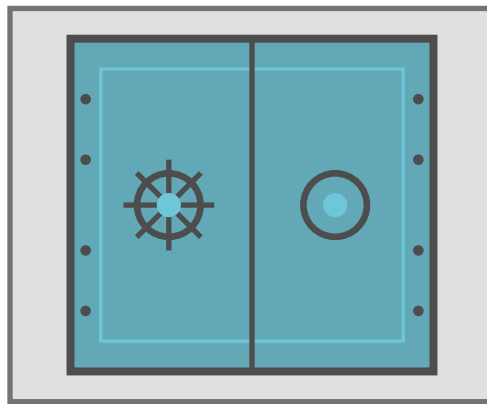


Criptografía y Seguridad

Criptografía Cifrado

Secreto perfecto · Seguridad computacional · Modos de bloque · CPA/CCA



Apunte integral — Teoría + Práctica

Clase 2 · Capítulos 2 y 3, *Introduction to Modern Cryptography* (Katz & Lindell)

Instituto Tecnológico de Buenos Aires (ITBA)

Documento elaborado a partir de las transcripciones de la clase teórica (Prof. Pablo Abad) y de la clase práctica, junto con las diapositivas oficiales de ambas.

Índice

1. Repaso: criptosistema y secreto perfecto	2
1.1. Secreto perfecto	2
2. One Time Pad (cifrado de Vernam)	3
2.1. Propiedad fundamental: el OTP tiene secreto perfecto	3
2.2. Las malas noticias del OTP	4
3. Ejercicio: clave NO aleatoria	5
4. Más allá del OTP: resultados teóricos	7
5. Seguridad computacional	7
6. Criptosistemas de flujo (stream ciphers)	8
6.1. Generadores pseudoaleatorios (PRG)	9
6.2. Ejemplo de PRG (resuelto)	10
7. Pruebas de seguridad e indistinguibilidad	11
7.1. Prueba de indistinguibilidad ante observadores (EAV)	11
7.2. Múltiples cifrados (Mul)	12
7.3. Los stream ciphers NO son seguros bajo múltiples cifrados	13
7.4. Necesidad de cifrado probabilístico	13
7.5. Evitar reutilizar la clave: nonce / IV	13
8. Ataques de texto plano escogido (CPA)	14
9. Primitivas de cifrado en bloque	15
9.1. Extensión y encadenamiento	16
10. Modos de operación de block ciphers	16
10.1. ECB — Electronic Code Book	16
10.2. CBC — Cipher Block Chaining	17
10.3. OFB — Output Feedback	18
10.4. CTR — Counter	18
10.5. CFB — Cipher Feedback	19
10.6. Seguridad de los modos	19
11. DES — Data Encryption Standard	20
11.1. Estructura de Feistel	20
11.2. La función de transformación F y la generación de subclaves	20
11.3. Claves débiles y semidébiles (clase práctica)	21
11.4. Evolución de DES	21
11.5. 3-DES	21
12. AES — Advanced Encryption Standard	21
13. Elección de criptosistemas y estado de un criptosistema	22
13.1. Criptosistemas recomendados	23
Material complementario (clase práctica)	23
Lectura recomendada	23

1 Repaso: criptosistema y secreto perfecto

Esta clase retoma los formalismos introducidos al final de la clase anterior y los profundiza: arranca repasando la definición de **criptosistema** y de **secreto perfecto**, demuestra que el *One Time Pad* (OTP) alcanza ese ideal, muestra por qué es impráctico, y de ahí construye toda la **seguridad computacional** moderna (criptosistemas de flujo, modos de bloque, DES/AES).

Criptosistema (terna de algoritmos)

Un criptosistema es una **terna de algoritmos** $\langle \text{Gen}, e, d \rangle$:

$\text{Gen} : () \rightarrow \mathcal{K}$ Generador de clave

$e : \mathcal{K} \times \mathcal{P} \rightarrow \mathcal{C}$ Cifrado

$d : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{P}$ Descifrado

Propiedad básica: para todo m y k válidos, $d_k(e_k(m)) = m$.

Conjuntos involucrados:

- $\mathcal{K}$ — espacio de claves: todas las claves posibles.
- $\mathcal{P}$ — espacio plano: todos los mensajes posibles.
- $\mathcal{C}$ — espacio cifrado: todos los mensajes cifrados posibles.

Aclaración del profe. El generador Gen “en los criptosistemas más comunes suele ser simplemente una función *no determinística* que toma una clave desde el espacio de claves con una distribución aleatoria uniforme”. Es decir, **elige una clave al azar**: nos da la idea de que la clave criptográfica es totalmente aleatoria. La terna establece la *relación* entre las funciones, pero **todavía no habla de seguridad**: ya se adelantó que **no hay una única definición de seguridad**.

1.1 Secreto perfecto

Secreto perfecto (Shannon)

Dado un criptosistema $\langle \text{Gen}, e, d \rangle$, posee **secreto perfecto** si para toda distribución de probabilidades en $\mathcal{M}$, cada mensaje m y cada mensaje cifrado c tal que $\Pr[C = c] > 0$:

$$\Pr[M = m \mid C = c] = \Pr[M = m]$$

En palabras: **conocer el texto cifrado no da ningún tipo de información extra sobre el texto plano** que no se conociera de antemano. A priori uno puede tener una *tabla de probabilidades* sobre los mensajes (p. ej. “hay un 90 % de chances de que el mensaje sea tal”); la ecuación dice que, conociendo el valor concreto que tomó el texto cifrado, esa tabla de probabilidades **no cambia**.

La paradoja del secreto perfecto

Notar que $c = e_k(m)$, así que las variables aleatorias discretas C y M son **dependientes** (hay una función que las relaciona). Sin embargo, la propiedad de secreto perfecto, interpretada probabilísticamente, dice que son **V.A.D.s independientes**.

El truco (explicación del profe): estas ecuaciones *no toman en cuenta las claves*. Las claves son las que introducen la capacidad de llegar a la independencia estadística *a pesar* de que

exista una función determinística que vincule M con C .

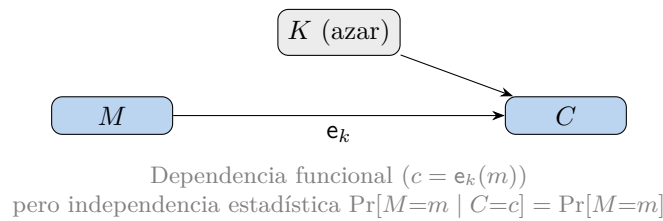


Figura 1: La clave aleatoria “rompe” la información que C podría revelar sobre M .

2 One Time Pad (cifrado de Vernam)

Anécdota histórica del profe. Shannon y, en paralelo, el investigador *Vernam* estudiaron si la condición de secreto perfecto era un ideal asintótico (inalcanzable) o realizable. Ambos llegaron casi independientemente a la misma conclusión: **sí es posible construir una función con esas características**, y es **la transformación más simple y tonta que se les pueda ocurrir**: el One Time Pad (criptosistema de Vernam, 1917).

One Time Pad — atribuido a Vernam (1917)

Gen : $k \leftarrow \{0, 1\}^n$, $n = |m|$ (clave aleatoria del tamaño del mensaje)

e : $e_k(m) = m \oplus k$

d : $d_k(c) = c \oplus k$

Cifrar y descifrar es un **XOR bit a bit** entre el mensaje y la clave.

El descifrado es igual de fácil porque, en base 2, el XOR es una operación **lineal** y **su propia inversa**: $(m \oplus k) \oplus k = m$. “Y ahí se acabó la magia.”

$$\begin{array}{rcl}
 m = & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0 & 0 & 1 & 0 & 1 & 1 & 1 & 0 \\
 k = & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 & 0 \\
 \hline
 c = & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 & 0 & 1 & 0 & 0
 \end{array} \quad c = m \oplus k$$

Figura 2: Ejemplo de la diapositiva: cada bit de c es el XOR del bit de m con el de k . (*Recreación de la diapositiva del OTP.*)

2.1 Propiedad fundamental: el OTP tiene secreto perfecto

Estrategia de la demostración (en dos partes)

Para cualquier par (m, c) de longitud n , con $|\mathcal{K}| = 2^n = N$, queremos probar $\Pr[M=m \mid C=c] = \Pr[M=m]$.

- **Lema (Parte 1):** mostrar que $\Pr[C = c] = \frac{1}{N}$, *independiente del mensaje*.
- **Parte 2:** usar la regla del producto / cambio de variable para concluir.

Demostración (1) — el lema $\Pr[C = c] = \frac{1}{N}$. La probabilidad de que el texto cifrado tome un valor concreto se obtiene sumando sobre *todas* las claves: para cada clave k , el texto cifrado vale c cuando ocurre la conjunción de un mensaje y esa clave que, al pasar por la función, dan c . Como la clave no puede tomar dos valores distintos a la vez, los eventos son **disjuntos** y la probabilidad total es la suma:

$$\Pr[C = c] = \sum_k \Pr[C = c \cap K = k].$$

Usando que $c = e_k(m) \iff M = c \oplus k$ (aplicando la función de descifrado) y que k y m **se eligen independientemente**:

$$\Pr[C = c \cap K = k] = \Pr[M = c \oplus k \cap K = k] = \Pr[M = c \oplus k] \cdot \Pr[K = k] = \Pr[M = c \oplus k] \cdot \frac{1}{N},$$

ya que en el OTP la clave es uniforme: $\Pr[K = k] = 1/N$. Entonces

$$\Pr[C = c] = \frac{1}{N} \sum_k \Pr[M = c \oplus k].$$

Como c es fijo y k recorre las N claves, $c \oplus k$ recorre **todos los mensajes posibles** (cada uno una sola vez, sin repetir). Por lo tanto la suma recorre todos los valores de M , y la suma de las probabilidades de una V.A.D. sobre todos sus valores es 1:

$$\Pr[C = c] = \frac{1}{N} \cdot 1 = \frac{1}{N}$$

Qué implica el lema. La probabilidad de que el cifrado tome un valor en particular **no depende** del mensaje que estábamos cifrando. Aun sabiendo a priori que el 90 % (¡o el 100 %!) de las veces el mensaje original es 0, cada posible valor de C sigue teniendo probabilidad $1/N$. Esto ya es una pista de que el OTP tiene secreto perfecto: viendo el texto cifrado solo, no aprendemos nada del texto plano.

Demostración (2) — conclusión. Partimos de la regla del producto para probabilidades condicionales:

$$\Pr[M = m \mid C = c] \cdot \Pr[C = c] = \Pr[C = c \cap M = m].$$

Como en el OTP $C = M \oplus K$, el evento $\{C = c \cap M = m\}$ equivale a $\{K = c \oplus m \cap M = m\}$. Por independencia de K y M :

$$\Pr[C = c \cap M = m] = \Pr[K = c \oplus m \cap M = m] = \Pr[K = c \oplus m] \cdot \Pr[M = m] = \frac{1}{N} \cdot \Pr[M = m].$$

Reemplazando $\Pr[C = c] = 1/N$ (del lema):

$$\Pr[M = m \mid C = c] \cdot \frac{1}{N} = \frac{1}{N} \cdot \Pr[M = m] \implies \Pr[M = m \mid C = c] = \Pr[M = m]$$

que es exactamente la definición de secreto perfecto. ■

Detalle fino (consulta de Martín en la teórica)

En el paso $\Pr[K = c \oplus m] = 1/N$ se está *salteando un paso*: como la probabilidad de que K tome cualquier valor constante es siempre $1/N$ (distribución uniforme), se puede asignar a $K = c \oplus m$ el valor $1/N$ sin importar m . De no ser uniforme la clave, K y M no serían independientes y la separación en producto no valdría.

2.2 Las malas noticias del OTP

Limitaciones del One Time Pad

1. **Tamaño de clave:** secreto perfecto $\Rightarrow |\mathcal{K}| \geq |\mathcal{C}|$. La clave debe ser *tan grande como el mensaje*.

2. **No reutilizar la clave:** si una clave se usa 2 veces,

$$c_1 = m_1 \oplus k, \quad c_2 = m_2 \oplus k \implies c_1 \oplus c_2 = m_1 \oplus m_2.$$

Se cancela la clave y se pierde el secreto perfecto.

3. **La clave seleccionada DEBE ser aleatoria.** ¿Cómo garantizar verdadera aleatoriedad?

Cuando una clave se reutiliza, $c_1 \oplus c_2 = m_1 \oplus m_2$. Como el XOR es lineal, las **propiedades estadísticas** de los mensajes se mantienen, así que reutilizar la clave convierte al OTP en **un criptosistema de sustitución** (un caso apenas más difícil que las sustituciones monoalfabéticas).

Sobre la aleatoriedad (profe). No existe un “generador mágico de números realmente aleatorios”; todo lo disponible son *aproximaciones* asintóticas: chips con generadores de hardware, ruido del micrófono, cadencia de tecleo, tiempos de *seek* de discos magnéticos, ruido eléctrico de un capacitor cargando/descargando (`/dev/random` de Linux mezclaba varias fuentes), decaimiento radiactivo. Si no tenemos un generador realmente aleatorio, **técnicamente no podríamos implementar el OTP** (nos saldríamos de una hipótesis de la demostración).

Usos históricos documentados (anécdotas del profe)

- **Segunda Guerra Mundial:** una variante de OTP *en papel*, en base 26/27, para cifrar letra a letra contra un cuaderno. Pusieron a 100 personas con máquinas de escribir a teclear “loco”, tomaron una letra de cada una, generaron una secuencia, la escribieron a mano en **dos cuadernos idénticos** y le dieron uno a cada parte.
- **Guerra Fría — el “teléfono rojo”:** la línea segura entre EE.UU. y la URSS estaba protegida con un OTP. La secuencia aleatoria se generó **midiendo la tasa de decaimiento radiactivo** dentro de un reactor; se imprimió, se puso en dos maletas esposadas a agentes, y habilitaba ~media hora de comunicación hablada **a prueba de manipulaciones**.
- **Por qué no sirve para e-commerce:** por cada *request* habría que generar una clave del tamaño del request y hacerla llegar por un canal aparte. Impráctico en el mundo comercial normal.

3 Ejercicio: clave NO aleatoria

Este ejercicio numérico muestra qué pasa cuando la clave **no** es aleatoria (tiene sesgo): se rompe el secreto perfecto y el atacante extrae información.

Enunciado. Clave no aleatoria sobre mensajes de 2 bits:

$$\Pr[k=00] = 0.3, \quad \Pr[k=01] = 0.1, \quad \Pr[k=10] = 0.4, \quad \Pr[k=11] = 0.2.$$

Distribución del texto plano:

$$\Pr[m=00] = 0.60, \quad \Pr[m=01] = 0.15, \quad \Pr[m=10] = 0.10, \quad \Pr[m=11] = 0.15.$$

Se obtiene un mensaje cifrado $C = 01$. ¿Qué probabilidad hay de que $M = 00$?

Notar que la clave 10 es *más probable* que las otras: si fuese aleatoria, cada valor tendría probabilidad 0.25. Queremos $\Pr[M=00 \mid C=01]$. Como M y C **no** son independientes, usamos la regla del producto:

$$\Pr[M=00 \mid C=01] \cdot \Pr[C=01] = \Pr[C=01 \mid M=00] \cdot \Pr[M=00].$$

Cálculo de $\Pr[C=01]$. Sumando sobre todas las claves (cada combinación m, k que produce $c = 01$):

$$\begin{aligned} \Pr[C=01] &= \sum_k \Pr[C=01 \cap K=k] = \sum_k \Pr[M=01 \oplus k] \cdot \Pr[K=k] \\ &= \Pr[M=01] \Pr[K=00] + \Pr[M=00] \Pr[K=01] \\ &\quad + \Pr[M=11] \Pr[K=10] + \Pr[M=10] \Pr[K=11] \\ &= 0.15 \cdot 0.3 + 0.6 \cdot 0.1 + 0.15 \cdot 0.4 + 0.1 \cdot 0.2 = \mathbf{0.185}. \end{aligned}$$

Cálculo de $\Pr[C=01 \mid M=00]$. Si el mensaje era 00, hay una **única** clave que produce $c = 01$, a saber $k = 01$ (pues $00 \oplus 01 = 01$):

$$\Pr[C=01 \mid M=00] = \Pr[K=01] = \mathbf{0.1}.$$

Juntando todo.

$$\Pr[M=00 \mid C=01] \cdot 0.185 = 0.1 \cdot 0.6 \implies \Pr[M=00 \mid C=01] = \frac{0.06}{0.185} = \boxed{0.32}.$$

Pero a priori $\Pr[M=00] = 0.6$. Como $0.32 \neq 0.6$, **no se cumple secreto perfecto**: aprendimos algo del texto plano. Concretamente, habiendo interceptado 01, es **menos probable de lo que creíamos** que el mensaje fuera 00.

Lectura del ataque (profe, respondiendo a Tomás/Santiago)

Si esto *tuviera* secreto perfecto, la respuesta sería 0.6 (ignorando el sesgo): conocer el cifrado no diría nada nuevo. El ataque simula un atacante que hizo **ingeniería inversa** de lo que se creía una secuencia aleatoria y la convirtió en una *no aleatoria*: **no necesita predecir la clave completa**, le alcanza con detectar un *sesgo* para corregir las probabilidades a su favor.

4 Más allá del OTP: resultados teóricos

Dos teoremas importantes sobre el secreto perfecto

1. **Cualquier criptosistema con secreto perfecto es reducible al OTP** (existe un isomorfismo: cualquier sistema con secreto perfecto es “otra forma, más enrevesada, de representar al OTP”).
2. **Cualquier sistema que NO sea reducible al OTP no posee secreto perfecto** (la contrarrecíproca).

Consecuencias: el secreto perfecto es *demasiado impráctico*; necesitamos **otras construcciones**. Esta es la razón de fondo por la cual *no hay una única definición de seguridad*.

5 Seguridad computacional

Secreto perfecto = Seguridad incondicional.

El secreto perfecto es **seguridad incondicional**: protege incluso contra un adversario con **capacidad de cómputo infinita**. Para escapar de sus limitaciones, relajamos dos cosas y pasamos a la **seguridad computacional**.

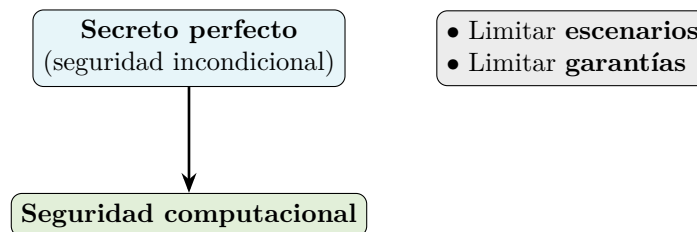


Figura 3: Para construir criptosistemas prácticos se “baja” del ideal incondicional.

Las dos relajaciones

1. **Garantizar seguridad solo contra adversarios “limitados”.** Se asume un **límite en los recursos del atacante** (especialmente el *tiempo*). Si alguien materializa el cómputo infinito, *game over*.
2. **Aceptar una pequeña probabilidad de éxito para el atacante.** Se deja de lado la **infalibilidad**: se acepta una probabilidad de quiebre tan pequeña que podamos “dormir tranquilos”.

Con estas dos condiciones, algo deja de ser *imposible* para ser *computacionalmente imposible*.

Por qué aceptar una probabilidad de éxito (analogía de la lotería, profe). Al salir del secreto perfecto aparece un sesgo entre textos cifrados $\Rightarrow$ el ataque por **fuerza bruta** siempre es posible. Como en la lotería: quien recorre todas las claves *garantiza* el quiebre, pero también está el afortunado que compra un único número y gana. Por eso hay que aceptar que un atacante con suerte puede quebrar un caso particular (no es un quiebre general).

Redefinición del esquema (clase práctica)

La práctica redefine el esquema privado $\pi = (\text{Gen}, \text{Enc}, \text{Dec})$ parametrizado por el nivel de seguridad n :

$$k \leftarrow \text{Gen}(1^n), \quad c \leftarrow \text{Enc}_k(m), \quad m := \text{Dec}_k(c),$$

y redefine el experimento de indistinguibilidad ante observadores como:

$$\Pr[\text{PrivK}_{A,\pi}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n),$$

donde la seguridad se exige solo frente a **adversarios eficientes** y con una **probabilidad de éxito despreciable**.

Adversario eficiente (PPT) y función despreciable (clase práctica)

Adversario eficiente / PPT.

Dado un nivel de seguridad n , se espera que el adversario corra algoritmos de orden $\text{PPT}(n)$ —*Probabilistic Polynomial Time*—: **tiempo de ejecución polinómico** en n (puede ser n^2 , n^{40} , no importa, pero *polinómico*). Esto descarta explícitamente al atacante que prueba las 2^n claves por fuerza bruta.

Función despreciable.

La probabilidad de éxito $\varepsilon(n)$ debe ser **despreciable** en n :

$$\varepsilon(n) \text{ es despreciable} \iff \lim_{n \rightarrow \infty} \varepsilon(n) < \frac{1}{n^k} \quad \forall k,$$

es decir, decrece más rápido que el inverso de cualquier polinomio.

Nivel de seguridad n (profe). Es la cota sobre el orden de magnitud de la complejidad del adversario; en la práctica coincide con la **cantidad de bits de la clave** (con clave de 128 bits, $n = 128$). Hoy, juntando todas las computadoras del planeta, se pueden hacer $\sim 2^{76}$ operaciones por año; con 2^{128} claves haría falta **toda la vida del universo**. Por eso ponemos cota *polinómica*: la fuerza bruta sistemática es “fácil” de volver impráctica.

La cuña de las computadoras cuánticas: para ciertos problemas existen algoritmos exponenciales en una máquina clásica que pasan a ser *polinómicos* en una cuántica; ese cambio de categoría es lo que hace tambalear a algunos criptosistemas.

6 Criptosistemas de flujo (stream ciphers)

Los criptosistemas de flujo **toman la idea del OTP** pero relajan su condición más incómoda: **intercambian la clave del OTP por la salida de un generador pseudoaleatorio G** .

Criptosistema de flujo

$$\text{Gen} : k \leftarrow \mathcal{K}$$

$$\text{e} : \text{e}_k(m) = G(k) \oplus m$$

$$\text{d} : \text{d}_k(c) = G(k) \oplus c$$

La gran diferencia: $|\mathcal{K}| \lll |\mathcal{M}|$.
Por ejemplo: $|\mathcal{K}| = 2^{128}$, $|\mathcal{M}| = |\mathcal{K}|^{128}$.

La idea (profe): se usa una clave *pequeña* (típicamente 128 bits) como **semilla** de un generador que produce tantos bits como tenga el mensaje, y se hace el XOR como en el OTP. “Si queremos mandar una novela, la podemos cifrar con una clave de 8 letras.” Cifrar/descifrar siguen siendo invertibles. Pero ya **no** aplica el marco del secreto perfecto: hace falta otra definición de seguridad.

6.1 Generadores pseudoaleatorios (PRG)

Los generadores pseudoaleatorios:

- Son **algoritmos determinísticos**.
- Expanden una entrada llamada **semilla** (*seed*).
- La **salida parece aleatoria**.
- Con la **misma semilla** generan la **misma secuencia**.

El ejemplo de la diapositiva es un **LFSR** (Linear Feedback Shift Register): un registro de desplazamiento cuya realimentación es el XOR de ciertas posiciones (*taps*).

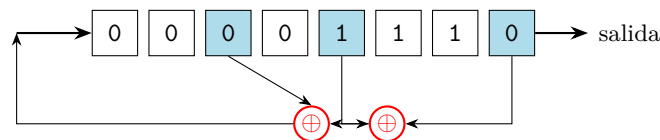


Figura 4: LFSR: la realimentación (entrada izquierda) es el XOR de las posiciones sombreadas (*taps*); en cada paso se desplaza y emite un bit. (*Recreación de la diapositiva.*)

Generador pseudoaleatorio — definición formal

Sea $D = \{f : \{0, 1\}^n \rightarrow \{0, 1\}\}$ una **familia de funciones** (clasificadores o *distinguishers*: dada una secuencia, la clasifican en 0 o 1).

$G : \{0, 1\}^s \rightarrow \{0, 1\}^n$, con $s < n$, es un **generador pseudoaleatorio** respecto de D si:

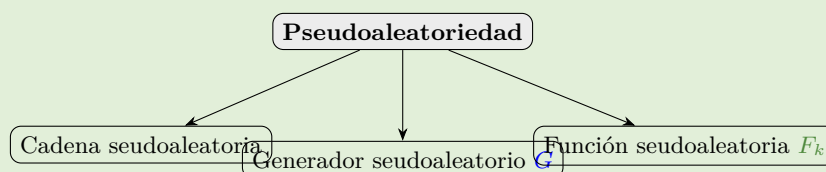
$$\text{para toda } f \in D : \quad \Pr [f(G(r^s)) \neq f(r^n)] = \varepsilon,$$

donde r^s es la semilla, r^n es una **secuencia realmente aleatoria**, y ε es un **valor despreciable**.

Idea (profe). Un PRG es determinístico pero su salida debe *parecer* aleatoria. Formalmente: si juntamos *todas* las funciones clasificadoras que existan, G es pseudoaleatorio si para ninguna de ellas la salida del generador se clasifica distinto que una secuencia realmente aleatoria, salvo con probabilidad despreciable. Ejemplo de clasificador simple: contar ceros y unos y devolver 1 si hay un sesgo (más ceros que unos) y 0 si no.

Jerarquía de pseudoaleatoriedad (clase práctica)

La práctica organiza la **pseudoaleatoriedad** en tres objetos:



Cadena pseudoaleatoria.

Parece una cadena con distribución uniforme (aleatoria).

Generador pseudoaleatorio G .

Algoritmo determinístico que recibe una semilla s pequeña y aleatoria y la expande a una r de longitud mayor y pseudoaleatoria = **stream cipher**. Ej.: **RC4**, **LFSR**. Formalmente $r := G(s)$ con $|s| = n$ y $|r| = \ell(n) > n$. La semilla s es el secreto.

Función pseudoaleatoria F_k .

Función indistinguible de una elegida al azar del conjunto $F : \{0,1\}^n \rightarrow \{0,1\}^n = \mathbf{block ciphers}$ (ver §9).

OTP redefinido con G (clase práctica) — seguro ante *eavesdropping*

$$k \leftarrow \text{Gen}(1^n), \quad c := G(k) \oplus m, \quad m := G(k) \oplus c, \quad |k| = n, \quad |m| = \ell(n) > n.$$

Esquema seguro ante *eavesdropping* para mensajes de longitud fija:

$$\Pr[\text{PrivK}_{A,\text{OTP}}^{\text{eav}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n).$$

Pero el OTP (de flujo) NO es seguro para múltiples mensajes (vectores de mensajes) si se reutiliza la clave.

6.2 Ejemplo de PRG (resuelto)

Enunciado. Sea $s \in \{1, \dots, 10\}$ y

$$G(s) = \{G_0 \text{ mód } 2, G_1 \text{ mód } 2, G_2 \text{ mód } 2, \dots\}, \quad G_0 = s, \quad G_i = (G_{i-1} \cdot 3 + 1) \text{ mód } 11.$$

Generar 5 bits con $s = 2$ y con $s = 6$.

Con $s = 2$. Iteramos $G_i = (3G_{i-1} + 1) \text{ mód } 11$ y tomamos paridad:

i	0	1	2	3	4
G_i	2	7	0	1	4
$G_i \text{ mód } 2$	0	1	0	1	0

($G_1 = 3 \cdot 2 + 1 = 7$; $G_2 = 3 \cdot 7 + 1 = 22 \text{ mód } 11 = 0$; $G_3 = 3 \cdot 0 + 1 = 1$; $G_4 = 3 \cdot 1 + 1 = 4$.)
Salida: 01010.

Con $s = 6$.

i	0	1	2	3	4
G_i	6	8	3	10	9
$G_i \text{ mód } 2$	0	0	1	0	1

($G_1 = 3 \cdot 6 + 1 = 19 \text{ mód } 11 = 8$; $G_2 = 3 \cdot 8 + 1 = 25 \text{ mód } 11 = 3$; $G_3 = 3 \cdot 3 + 1 = 10$; $G_4 = 3 \cdot 10 + 1 = 31 \text{ mód } 11 = 9$.) Salida: 00101.

Semillas distintas $\Rightarrow$ secuencias distintas; misma semilla $\Rightarrow$ misma secuencia. (Este G de juguete es un generador congruencial lineal: **no** es criptográficamente seguro, sirve solo de ejemplo.)

7 Pruebas de seguridad e indistinguibilidad

Pruebas de seguridad

Prueban características de un criptosistema. Son una **serie de pasos que ejecutan un algoritmo** (que representa un ataque). El atacante **gana o pierde** la prueba; se puede **repetir múltiples veces** $\Rightarrow$ interesa la **probabilidad de éxito** del atacante.

Aclaración del profe. A diferencia de la demostración del OTP (matemática), las pruebas estadísticas son “un **juego de guerra**” entre dos actores: el criptosistema (un algoritmo / terna de algoritmos) y el atacante (*cualquier* algoritmo). Repitiendo el juego muchas veces se obtiene una probabilidad de éxito y, a partir de ella, una noción *condicional probabilística* de seguridad.

7.1 Prueba de indistinguibilidad ante observadores (EAV)

Eavesdropping Indistinguishability test: $\text{Eav}_{A,\Pi}$

Dado un adversario A y un criptosistema Π :

1. A genera m_0 y m_1 arbitrariamente.
2. Se genera una clave $k \leftarrow \mathcal{K}$.
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = \text{e}_k(m_b)$.
5. A emite $b' \in \{0, 1\}$.

$\text{Eav}_{A,\Pi} = 1$ si $b = b'$ (A gana).

Π es **indistinguible** si $\Pr[\text{Eav}_{A,\Pi} = 1] = \frac{1}{2} + \varepsilon$, con ε despreciable.

Versión parametrizada (nivel de seguridad): adversario $A(n)$, criptosistema $\Pi(n)$, y $\Pr[\text{Eav}_{A,\Pi} = 1] = \frac{1}{2} + \varepsilon(n)$.

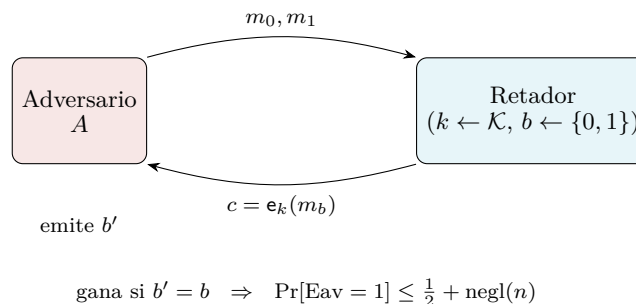


Figura 5: Experimento $\text{PrivK}_{A,\Pi}^{\text{eav}}$: el adversario elige los dos mensajes y debe distinguir cuál se cifró. (*Recreación de la diapositiva.*)

Por qué $\frac{1}{2}$ (profe). Aunque A no sepa nada del criptosistema, está obligado a emitir b' . Si tira al azar (o siempre 0, o siempre 1), **acierta la mitad de las veces** en promedio. La prueba reduce “¿puede el adversario extraer información?” a su **mínima expresión** y en el **mejor escenario para el atacante**: solo hay 2 mensajes posibles y *los elige el propio atacante*. Si A puede hacer algo mejor que adivinar (sesgar la respuesta por encima de 0.5), es que **logró extraer información**.

Nivel de seguridad y la naturaleza de la prueba (profe)

- Como es probabilística, la probabilidad “nunca va a ser exactamente 0.5”; por eso aparece el ε (debe estar a una cantidad *despreciable* de $\frac{1}{2}$).
- La prueba **no es absoluta**: pone a prueba el criptosistema contra *un* algoritmo adversario concreto. Que no *conozcamos* un adversario ganador no significa que no exista. En la práctica, “indistinguible ante *eavesdropping*” significa: con **todo el conocimiento colectivo** de funciones adversariales inventadas hasta hoy, ninguna gana la prueba.
- Queremos funciones **todo terreno** (que mantengan la propiedad para todo tipo de mensajes), no funciones de uso hiperspecífico (“buenísima salvo si tus mensajes son numéricos”), porque los sistemas mutan y los supuestos cambian.

Teorema: stream cipher con PRG es IND-EAV (slide + práctica)

Teorema. Si $G(\cdot)$ es un generador pseudoaleatorio, entonces el criptosistema de flujo $\text{Enc}_k(m) = G(k) \oplus m$ es **indistinguible ante observadores**.

Ejercicio (slide). Demostrar que *si es posible distinguir $G(\cdot)$ de una secuencia aleatoria, un criptosistema de flujo basado en G no pasa la prueba EAV*.

Idea de la demostración por reducción (profe): si G **no** es PRG entonces existe un distinguidor f ; usando f se construye un adversario A que gana la prueba EAV. Contrarrecíproca: si G es PRG, nadie gana $\Rightarrow$ el criptosistema es IND-EAV. (Demostración completa en Katz & Lindell.)

7.2 Múltiples cifrados (Mul)

Multiple message eavesdropping test: $\text{Mul}_{A,\Pi}$

Dado un adversario A y un criptosistema Π :

1. A genera dos *vectores* de mensajes $(m_{00}, m_{01}, \dots, m_{0i})$ y $(m_{10}, m_{11}, \dots, m_{1i})$.
2. Se genera una clave $k \leftarrow \mathcal{K}$.
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $(c_0, c_1, \dots, c_i)$, donde $c_j = \text{Enc}_k(m_{bj})$.
5. A emite $b' \in \{0, 1\}$.

$\text{Mul}_{A,\Pi} = 1$ si $b = b'$. Π es indistinguible bajo múltiples cifrados si $\Pr[\text{Mul}_{A,\Pi} = 1] = \frac{1}{2} + \varepsilon$.

7.3 Los stream ciphers NO son seguros bajo múltiples cifrados

Ataque que gana la prueba Mul

Los criptosistemas de flujo **no** son seguros bajo múltiples cifrados, por la misma razón que el OTP con reúso de clave (¡no es casualidad!):

$$c_1 = m_1 \oplus G(s), \quad c_2 = m_2 \oplus G(s) \implies c_1 \oplus c_2 = m_1 \oplus m_2.$$

Solución (ataque explícito de la slide)

Definir un algoritmo A que gane la prueba de múltiples cifrados:

$$A \rightarrow (m_{00} = 0 \cdots 0, m_{01} = 0 \cdots 0), (m_{10} = 0 \cdots 0, m_{11} = 1 \cdots 1).$$

A obtiene c_1, c_2 y calcula $X = c_1 \oplus c_2 = m_1 \oplus m_2$:

- Si $X = 0 \cdots 0 \rightarrow$ los dos mensajes eran iguales $\rightarrow$ responde $b' = 0$.
- Si no $\rightarrow$ eran distintos $\rightarrow$ responde $b' = 1$.

Gana siempre. Como en el primer vector ambos mensajes son iguales ($0 \cdots 0$), $c_1 \oplus c_2 = 0$; en el segundo son distintos ($0 \cdots 0$ vs. $1 \cdots 1$), $c_1 \oplus c_2 \neq 0$.

¿El atacante extrae información “real”? (pregunta de Tomás)

Tomás: ganaste la prueba, pero ¿extraes información más allá de notar que se usa la misma clave? *Profe:* en el peor caso, el atacante puede saber si **dos mensajes se repiten**. Ej.: un sensor que transmite temperatura cifrada con clave fija; sin recuperar el valor, el atacante puede deducir que “la temperatura de ahora es la misma que hace 15 min”. **Una debilidad teórica vuelve el ataque muy plausible;** es un tema de intereses (no la usarías para la cuenta bancaria de una corporación).

7.4 Necesidad de cifrado probabilístico

Conclusión clave de la materia

Si una función de cifrado es determinística, NO es segura bajo múltiples cifrados. El adversario anterior aplica a **cualquier** criptosistema donde $e_k(x)$ es constante (mapea siempre al mismo lugar).

$\Rightarrow$ **No se puede construir seguridad criptográfica seria con funciones determinísticas.** (Este error ha costado “miles de millones de dólares” en la historia.) A partir de acá, todo lo que veamos será **no determinístico**.

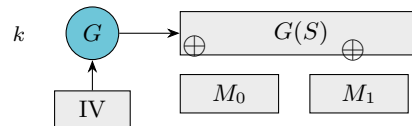
7.5 Evitar reutilizar la clave: nonce / IV

Se agrega un valor a la función generadora. **Ese valor no debe repetirse para una misma clave** (se lo llama **nonce** o **IV** —vector de inicialización—). **Dos formas:**

IV / Nonce: diferencia con la clave

Un **IV** es una secuencia arbitraria de bits elegida *aleatoriamente*. La única diferencia conceptual con una clave es que el IV es **información pública**: viaja con el texto cifrado (“lo publicamos en Internet”). Sirve para introducir el **no determinismo** que permite reutilizar la clave de forma segura.

Modo sincronizado (un único IV)



Modo no sincronizado (un IV por mensaje)

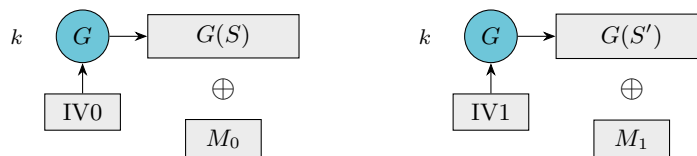


Figura 6: Sincronizado: un IV mezclado con k como semilla. No sincronizado: un IV nuevo y aleatorio por mensaje (el IV viaja como parte del ciphertext). (*Recreación de las diapositivas.*)

El catch (profe). Como el IV es necesario para descifrar (se requiere la misma semilla para reconstruir la secuencia), el texto cifrado debe **incluir el IV**. Esto rompe la simetría de tamaños: a un mensaje de 100 bytes se le agregan, p. ej., 16 bytes de IV. En el **modo sincronizado** (asincrónico en la práctica) se emite un IV por mensaje (bueno para mensajes grandes); en otro modo se piensa la transmisión como un **continuo** y se sigue usando el generador desde donde se quedó (sin emitir IV en cada mensaje). Ambos logran el no determinismo.

8 Ataques de texto plano escogido (CPA)

Chosen Plain Text indistinguishability: $CPA_{A,\Pi}$

Dado un adversario A y un criptosistema Π :

1. Se genera una clave $k \leftarrow \mathcal{K}$.
2. A obtiene **acceso al oráculo** $f(x) = e_k(x)$ y genera (m_0, m_1) .
3. Se genera $b \leftarrow \{0, 1\}$.
4. A recibe $c = e_k(m_b)$ (*challenge ciphertext*).
5. A sigue con acceso al oráculo y emite $b' \in \{0, 1\}$.

$CPA_{A,\Pi} = 1$ si $b = b'$. Π es **CPA-Secure** si $\Pr[CPA_{A,\Pi} = 1] = \frac{1}{2} + \varepsilon$, con ε despreciable.

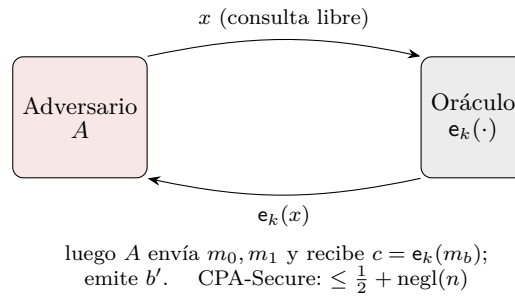


Figura 7: En CPA el adversario tiene un **oráculo de cifrado**: puede cifrar todos los textos que quiera antes y después de recibir el desafío. (*Recreación de la diapositiva.*)

Propiedades CPA (slide)

- Un criptosistema **determinístico no puede ser CPA-Secure** (el oráculo lo delata: ciframos m_0 y comparamos).
- Si un criptosistema es CPA-Secure **para un mensaje**, también lo es **para múltiples**.
- Un criptosistema CPA-Secure pero de **tamaño limitado** (cifra hasta n bits) puede **extenderse arbitrariamente**: con $m = m_0 \| m_1 \| \dots \| m_i$ ($|m_j| = n$ bits),

$$\text{Enc}_k(m) = \text{Enc}_k(m_0) \| \text{Enc}_k(m_1) \| \dots \| \text{Enc}_k(m_i).$$

9 Primitivas de cifrado en bloque

Primitiva de cifrado en bloque (block cipher)

Definida para mensajes de **tamaño FIJO**:

$$\mathcal{K} = \{0, 1\}^n, \quad \mathcal{C} = \mathcal{P} = \{0, 1\}^b,$$

$$\text{Gen} : k \leftarrow \mathcal{K}, \quad \text{Enc} : e_k(m) = c, \quad \text{Dec} : d_k(c) = m.$$

Son funciones pseudoaleatorias (F_k). Podrían “usarse” como criptosistemas, **pero son determinísticas** $\rightarrow$ **no pasan la prueba Mul**. Se combinan con **mecanismos de encañamiento** para formar criptosistemas seguros.

Esquema $\pi(\text{Gen}, \text{Enc}, \text{Dec})$ con función pseudoaleatoria F_k (clase práctica)

$$\text{Gen} : k \leftarrow \{0, 1\}^n \quad (|k| = n)$$

$$\text{Enc} : \text{para } m \text{ con } |m| = n, \text{ se elige } r \leftarrow \{0, 1\}^n; \quad c := \langle r, F_k(r) \oplus m \rangle$$

$$\text{Dec} : \text{dado } \langle r, s \rangle, \quad m := F_k(r) \oplus s$$

Esquema seguro ante CPA (Chosen Plaintext Attack): $\Pr[\text{PrivK}_{A,\pi}^{\text{CPA}}(n) = 1] \leq \frac{1}{2} + \text{negl}(n)$.

La clave del esquema es el r *aleatorio por mensaje*: introduce el no determinismo que el block cipher puro (determinístico) no tiene.

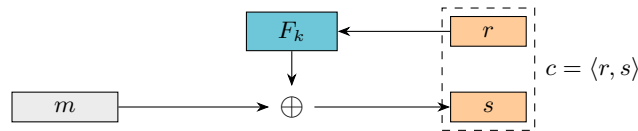


Figura 8: Cifrado CPA-seguro a partir de una PRF F_k y un r aleatorio por mensaje: $c = \langle r, F_k(r) \oplus m \rangle$. (Recreación de la diapositiva de la práctica.)

9.1 Extensión y encadenamiento

¿Y si el mensaje es más chico que el bloque? Se lo extiende sistemáticamente (*padding*):

- **Simple Pad:** completar con ceros. Es necesario conocer el tamaño real del mensaje.
- **DES Pad:** agregar un bit 1 y luego bits en 0. Puede agregar un bloque completo de padding.

¿Y si el mensaje es más grande que el bloque? Se divide en bloques, se extiende el último y se transforma cada bloque según algún **modo de encadenamiento**. El objetivo de los modos es **extender una primitiva de bloque a mensajes mayores a su tamaño**; hay diversos modos con distintas propiedades (no todos aplican a cada problema).

10 Modos de operación de block ciphers

10.1 ECB — Electronic Code Book

ECB —

Trata el mensaje original como un **conjunto de bloques independientes**; aplica la permutación pseudoaleatoria sobre cada bloque por separado.

$$m = m_1 m_2 \dots m_l, \quad c = \langle F_k(m_1), F_k(m_2), \dots, F_k(m_l) \rangle, \quad \text{Dec: } F_k^{-1}.$$

NO es CPA-Secure (es determinístico: bloques iguales $\rightarrow$ cifrados iguales, revela patrones).

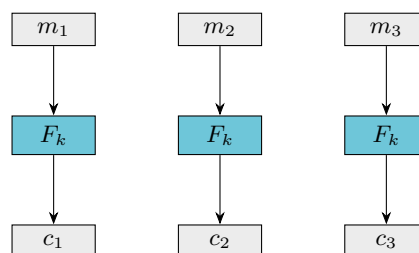


Figura 9: ECB: cada bloque se cifra de forma independiente. (Recreación de la diapositiva.)

10.2 CBC — Cipher Block Chaining

CBC

Usa la salida de un bloque como entrada para el próximo. Disminuye el traspaso de información. **Requiere un IV ALEATORIO.**

$$m = m_1 m_2 \dots m_l : \quad c_0 = \text{IV}, \quad c_i = F_k(c_{i-1} \oplus m_i).$$

El IV va en plano. **Es seguro frente a CPA** (con IV aleatorio); su contra es la **encriptación secuencial**.

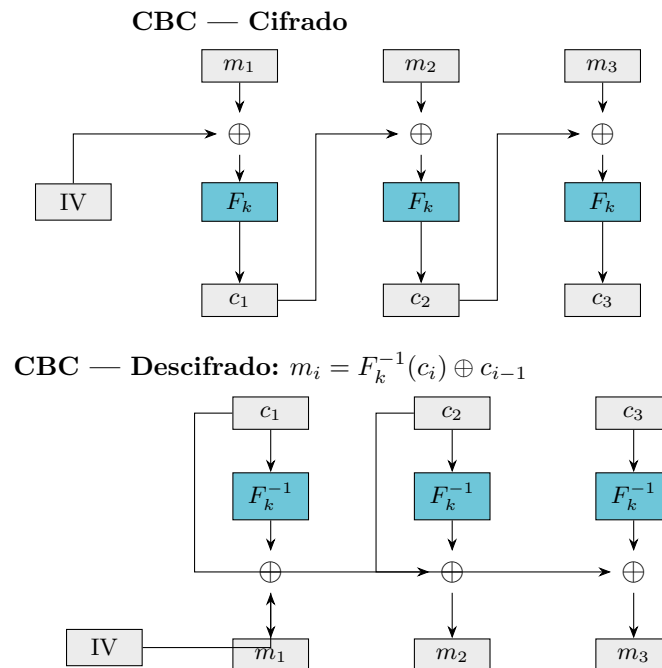


Figura 10: CBC: en cifrado cada bloque se XOR-ea con el cifrado anterior antes de pasar por F_k ; en descifrado se aplica F_k^{-1} y se XOR-ea con el cifrado anterior. (*Recreación de las diapositivas.*)

Propagación de error en CBC

Como $m_i = F_k^{-1}(c_i) \oplus c_{i-1}$, un error de transmisión en **un bloque cifrado** c_i afecta exactamente a **dos bloques de texto plano**: el propio m_i (que se corrompe por completo) y m_{i+1} (donde el error se traslada bit a bit vía el XOR con c_i). Los demás bloques se recuperan correctamente: *el error no se propaga más allá de dos bloques*.

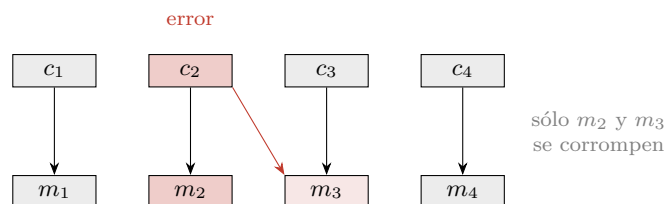


Figura 11: Un bit erróneo en c_2 corrompe m_2 (completo) y m_3 (bit a bit). El resto queda intacto.

10.3 OFB — Output Feedback

OFB

Construye una **transformación de flujo a partir de una de bloque**; usa **solo** la función Enc (menor complejidad para dispositivos embebidos). Permite **calcular los bits de la transformación por adelantado**.

$$r_0 = \text{IV}, \quad r_i = F_k(r_{i-1}), \quad c_i = r_i \oplus m_i. \quad (\text{IV en plano.})$$

Seguro frente a CPA, no requiere F_k biyectiva, los r_i se pueden precalcular; su contra es la **encriptación secuencial**.

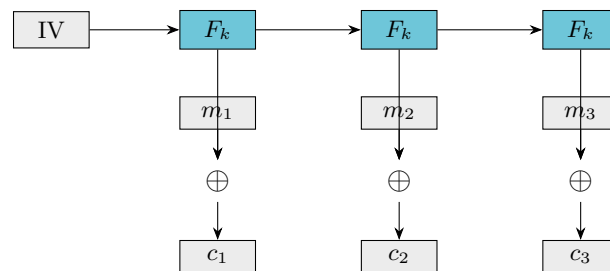


Figura 12: OFB: la cadena de claves $r_i = F_k(r_{i-1})$ se genera con solo Enc y se XOR-ea con el plano. (Recreación de la diapositiva.)

10.4 CTR — Counter

CTR (Counter Code Mode)

Utiliza un **contador** para generar la secuencia de bits de clave; permite **acceso aleatorio** y es muy útil con **procesamiento paralelo**.

$$\text{Ctr} = \text{IV}, \quad r_i = F_k(\text{Ctr} + i) \text{ (suma módulo } 2^n), \quad c_i = r_i \oplus m_i.$$

El contador se elige en forma aleatoria. **Seguro frente a CPA** (si no se repite (k, nonce)), **no requiere F_k biyectiva**, el stream se puede precalcular, permite cifrado/descifrado **en paralelo** y descifrar el bloque i -ésimo por separado.

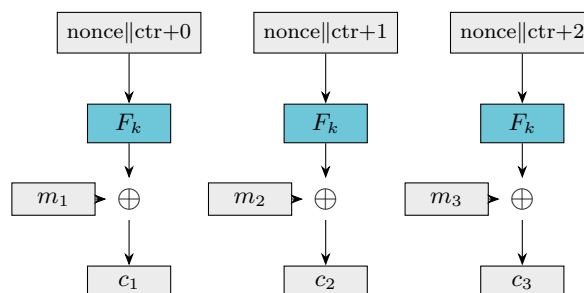


Figura 13: CTR: cada bloque cifra un contador independiente $\Rightarrow$ totalmente paralelizable y con acceso aleatorio. (Recreación de la diapositiva.)

10.5 CFB — Cipher Feedback

CFB

Usa solo la función **Enc** y permite generar una transformación de **flujo** a partir de una de bloque (menor complejidad para embebidos).

$$r_0 = \text{IV}, \quad c_1 = (F_k(r_0) \gg s) \oplus m_1, \quad r_i = (r_{i-1} \ll s) \mid c_{i-1}, \quad c_i = (F_k(r_i) \gg s) \oplus m_i,$$

donde s es el número de bits. Si $s = 1$, **equivale a un stream cipher**. Es seguro frente a CPA con IV aleatorio.

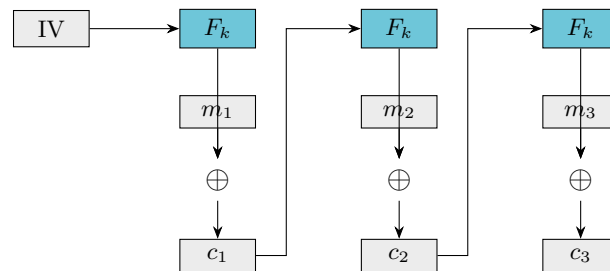


Figura 14: CFB: el cifrado anterior realimenta la entrada de F_k del siguiente bloque. (*Recreación de la diapositiva.*)

10.6 Seguridad de los modos

Seguridad de cifrado por bloques (resumen)

Las pruebas son **por reducción** a propiedades de la primitiva subyacente: requieren que la primitiva se comporte como una **función pseudoaleatoria**, es decir, que no sea posible distinguir $f(x) = \text{Enc}_k(x)$ de una función tomada al azar del conjunto de funciones del mismo dominio. Si la primitiva es PRF:

- **CBC** es CPA-Secure con IVs aleatorios.
- **OFB**, **CFB** son CPA-Secure con IVs aleatorios.
- **CTR** es CPA-Secure si no se repite (k, nonce) .

Salvedad: no está demostrado que existan las funciones pseudoaleatorias.

Modo	CPA-Secure	F_k biyectiva	Paralelizable	Precálculo del stream
ECB	NO	sí	sí	—
CBC	sí (IV aleat.)	sí (F_k^{-1})	cifrado: no	no
OFB	sí (IV aleat.)	no	no	sí
CTR	sí (k, nonce únicos)	no	sí	sí
CFB	sí (IV aleat.)	no	descifrado: sí	no

11 DES — Data Encryption Standard

DES

- Método desarrollado por **IBM**, adoptado por el gobierno de EE.UU. como estándar para usos no militares.
- Fue la **primera función de cifrado pública apoyada por un gobierno**; alcanzó uso comercial masivo.
- Trabaja con textos planos binarios. **Entrada: 64 bits. Clave: 64 bits (56 efectivos). Salida: 64 bits.**

11.1 Estructura de Feistel

DES se basa en una **red de Feistel**: se parte el mensaje en dos mitades, se transforma una mitad con parte de la clave, se intercambian las dos mitades de lugar, y se repiten esos tres pasos (una ronda) **16 veces**.

Caja Feistel

$$L_i = R_{i-1}, \quad R_i = L_{i-1} \oplus F(R_{i-1}, K_i).$$

Ventaja clave: el **descifrado usa la misma estructura** (con las subclaves en orden inverso), aunque F no sea invertible.

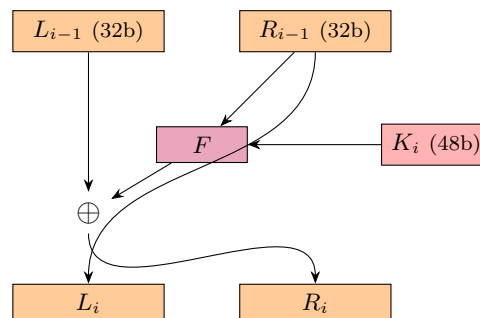


Figura 15: Una ronda de Feistel (DES): R_{i-1} pasa por F con la subclave K_i , se XOR-ea con L_{i-1} , y las mitades se intercambian. (*Recreación de la diapositiva.*)

11.2 La función de transformación F y la generación de subclaves

Función $F(R_{i-1}, K_i)$ — tres pasos

1. **E** — **Expansión**: de 32 a 48 bits.
2. **S_x** — **Sustituciones (S-boxes)**: 8 cajas que llevan de 6 a 4 bits.
3. **P** — **Permutación** (P-box).

Generación de subclaves

- **PC1**: selección de 56 bits (los 8 restantes son de *paridad*).
- **PC2**: separación en dos mitades y generación de cada subclave (24 bits de cada mitad → 48 bits).

- $\lll$: desplazamientos a nivel de bits entre rondas.

11.3 Claves débiles y semidébiles (clase práctica)

Claves débiles y semidébiles

Claves débiles.

$E_k(m) = D_k(m)$, o sea $E_k(E_k(m)) = m$ (el cifrado es su propia inversa). En lugar de generar 16 subclaves distintas, generan **una sola**.

Claves semidébiles.

Vienen *de a pares*: $E_{k_x}(E_{k_y}(m)) = m$. En lugar de 16 subclaves distintas, generan 2 o 4.

11.4 Evolución de DES

- **Nivel teórico de seguridad: 56 bits** (fuerza bruta: probar 2^{56} claves).
- Generó **desconfianza** por cuestiones de diseño confidenciales (las sustituciones / S-boxes).
- **1990 — Criptoanálisis diferencial**: ataca DES en 2^{47} intentos. Gran parte del diseño de DES *disminuye* el impacto de este ataque (otros sistemas fueron quebrados de inmediato).
- **1992 — Criptoanálisis lineal**: ataca DES en 2^{43} intentos.

11.5 3-DES

3-DES (variante fuerte de DES)

Selecciona 3 claves independientes y calcula

$$c = \text{Enc}_{k_1}(\text{Dec}_{k_2}(\text{Enc}_{k_3}(p))).$$

- Brinda seguridad del orden de **112 bits** (menor a 168 por el ataque *meet-in-the-middle*).
- Es **immune** a criptoanálisis diferencial y lineal.
- Es **mucho más lenta** que DES ($\times 3$).

12 AES — Advanced Encryption Standard

AES

- **Reemplazo de DES**, seleccionado mediante un **concurso internacional abierto** (5 años).
- Orientado a bloques de **128 bits**; clave de **128, 192 o 256 bits**.
- Es la **primitiva de cifrado recomendada** en la actualidad.

Esquema de funcionamiento — 4 etapas por ronda

Byte Sub.

Sustitución de valores según una tabla derivada de invertir una matriz en el campo $GF(2^8)$.

Shift Row.

Permutación de bits.

Mix Column.

Transformación lineal invertible de cada byte en función de los 4 que forman el mismo grupo.

Add Round Key.

XOR con la parte de la clave derivada para la ronda en cuestión.

Variaciones (asimetrías): la 1.^a ronda solo tiene *Add Round Key*; la **última ronda** no tiene *Mix Column*.

Round keys (128 bits). Los primeros 16 bytes corresponden a la clave original. Para los siguientes 16 bytes: **(a)** generar máscara — tomar los últimos 4 bytes y rotarlos 8 bits a la derecha, aplicar *ByteSub* a cada byte, y al byte menos significativo XOR con 2^i (siendo i el número de grupo de bytes a generar); **(b)** generar grupos de 4 bytes — los primeros 4: máscara XOR key-bytes generados 16 posiciones antes; los siguientes: XOR entre los 4 bytes generados y los generados 16 posiciones antes.

13 Elección de criptosistemas y estado de un criptosistema

Criptosistemas en proyectos

Se considera **mala práctica desarrollar un criptosistema nuevo** para un proyecto. Existen funciones estudiadas por años hasta alcanzar un nivel de confianza adecuado, y continuamente se publican avances que afectan la seguridad de funciones existentes. La regla es **reusar primitivas estándar y bien estudiadas**.

Estado de un criptosistema

Seguro.

Cumple con las expectativas de su modelo de seguridad.

Debilitado.

Existen adversarios con probabilidades no despreciables de éxito, pero el esfuerzo es muy alto (ej. décadas) o las condiciones muy difíciles (ej. disponer de 2^{80} mensajes).

Quebrado.

Existen adversarios con probabilidades no despreciables de éxito en **tiempos practicables**.

Escenarios: un criptosistema puede estar *seguro y quebrado al mismo tiempo*: hay múltiples pruebas de seguridad y cada una prueba un escenario diferente.

13.1 Criptosistemas recomendados

Criptosistemas de flujo (funciones G). En desuso / quebrados: RC4 (https, WEP), CSS (DVDs), A5/1, A5/2 (GSM), E0 (Bluetooth). Recomendados:

Salsa20 : $\{0, 1\}^{128 \times 256} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^n$, $n = 2^{64} \cdot 2^9$

Rabbit : $\{0, 1\}^{128} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^n$, $n = 2^{128}$ (el $\{0, 1\}^{64}$ es el IV)

Criptosistemas de bloque. En desuso: DES: $\{0, 1\}^{56} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^{64}$. Otros:

IDEA : $\{0, 1\}^{128} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^{64}$ (IDEA-CBC, IDEA-CTR)

3DES : $\{0, 1\}^{112 \times 168} \times \{0, 1\}^{64} \rightarrow \{0, 1\}^{64}$ (3DES-CBC, 3DES-CTR)

AES : $\{0, 1\}^{128, 192 \times 256} \times \{0, 1\}^{128} \rightarrow \{0, 1\}^{128}$ (AES-CBC, AES-CTR)

AES es el recomendado para proyectos nuevos.

Cifrado de bloque — tamaños. Espacios de clave: > 64 bits (AES: 128/192/256). Espacio de mensajes: ≥ 128 bits.

Para comparar: partículas en el universo $\sim 2^{88}$ átomos; edad estimada del universo $\sim 2^{58}$ segundos.

Material complementario (clase práctica)

- Video sobre **LFSR**: <https://www.youtube.com/watch?v=vfq3onw-uiM>
- Video sobre **RC4**: <https://www.youtube.com/watch?v=G3HajuqYH2U>
- Video sobre **DES**: <https://www.youtube.com/watch?v=XwU0wqSHzyo>
- Apunte e implementación de DES (y una implementación de AES): *Material Didáctico / Extra*.
- Ejercicios a resolver: **5, 6, 7 y 8**.

Lectura recomendada

Capítulos 2 y 3

Introduction to Modern Cryptography

Katz & Lindell

El cap. 2 cubre secreto perfecto y OTP; el cap. 3, la seguridad computacional, generadores pseudoaleatorios, las pruebas EAV/CPA/CCA y los modos de operación.

*Apunte integral de la Clase 2 — Cifrado · Criptografía y Seguridad, ITBA.
Síntesis de teoría (transcripciones + diapositivas) y práctica.*